

CAN Simulator GUI

Dominik Maślak

April 25, 2026

Contents

1	Opis projektu	2
2	Funkcje	2
2.1	Narzędzia podstawowe	2
2.2	Wyszukiwanie i analiza	2
2.3	Wizualizacja i ML	2
2.4	Sieć i zdalny dostęp	3
2.5	Makra i DBC	3
2.6	Funkcje niszowe (Etap E)	3
2.7	Inne	3
3	Wymagania	3
4	Instalacja	4
4.1	Sklonowanie repozytorium	4
4.2	Utworzenie środowiska wirtualnego i instalacja zależności	4
4.3	Przygotowanie interfejsu CAN (opcjonalnie wirtualny)	4
5	Uruchomienie	5
6	Struktura projektu	5
7	Etapy rozwoju	7
8	Testy i CI	7
9	Licencja	7

1 Opis projektu

Zaawansowane narzędzie do analizy, symulacji i diagnostyki magistral CAN. Integruje odtwarzanie logów, sniffer, wyszukiwanie binarne, kreator ramek, analizę ML (nadzorowaną i nienadzorowaną), prognozowanie LSTM, korelację sygnałów, eksport do ASC/Parquet/MDF4, edytor DBC, generator ruchu, makra, tryb serwera TCP i wiele więcej.

Aplikacja stworzona w Python 3 z interfejsem Tkinter. Wykorzystuje SocketCAN (Linux) i bibliotekę `python-can` do komunikacji z magistralą.

2 Funkcje

2.1 Narzędzia podstawowe

- **Odtwarzanie pliku** – wczytaj log `candump` i odtwórz ramki z regulowanym interwałem.
- **Symulacja modułu** – definiuj cykliczne ramki (missing frames).
- **Ramki błędu** – generuj niestandardowe ramki błędów.
- **Tryb krokowy** – ręczne wysyłanie ramek krok po kroku.
- **Wysyłanie ręczne** – pojedyncze ramki z możliwością wysyłki cyklicznej.

2.2 Wyszukiwanie i analiza

- **Wyszukiwanie binarne** – szybkie lokalizowanie ramki powodującej zjawisko (z progresem graficznym).
- **Kreator wyszukiwania** – znajdowanie konkretnej ramki lub sekwencji.
- **Sniffer CAN** – podgląd ruchu na magistrali w czasie rzeczywistym z filtrowaniem i eksportem.

2.3 Wizualizacja i ML

- **Wykresy** – graficzna analiza zmian wartości bajtów.
- **Zaawansowane ML** – prognozowanie sygnałów LSTM, korelacja, eksplorator danych.
- **Analiza wzorców** – wykrywanie anomalii za pomocą autoenkodera (PyTorch).
- **Analiza ML** – klasyfikacja sesji, anomalie (Isolation Forest).
- **Anomalie CAN** – dodatkowa analiza anomalii.

2.4 Sieć i zdalny dostęp

- **Serwer TCP** – udostępnianie strumienia CAN przez sieć.
- **Zdalny monitoring** – podgląd stanu połączeń.
- **Generator ruchu** – własne sekwencje testowe.
- **Mostek vCAN** – łączenie interfejsów wirtualnych.
- **Nagrywanie sesji** – rejestrowanie ruchu do późniejszej analizy.

2.5 Makra i DBC

- **Makra** – nagrywanie/odtworzenie sekwencji akcji.
- **DBC Manager** – edytor plików DBC, przeglądarka sygnałów.

2.6 Funkcje niszowe (Etap E)

- **Wbudowana konsola Python** – dostęp do obiektu `app` w locie.
- **System pluginów** – automatyczne wykrywanie i ładowanie modułów z katalogu `plugins/`.
- **Emulator prostego ECU** – odpowiada na zapytania diagnostyczne OBD-II (`0x7DF` → `0x7E8`).
- **Zdalny eval** – możliwość wykonania kodu Pythona na serwerze przez TCP.

2.7 Inne

- Eksport/import projektów (`.csp`)
- Dzienniki logów i zapis sesji (JSON)
- Motywy kolorystyczne (jasny/ciemny)
- Skróty klawiszowe (F5–F7, Ctrl+S, Ctrl+O)

3 Wymagania

- **System operacyjny:** Linux z obsługą SocketCAN (jądro z modułami `can`, `vcan`).
- **Python:** 3.8 lub nowszy (testowane na 3.10–3.13).
- **Zależności systemowe:**
 - `can-utils` (opcjonalnie, do `candump` i `cansniffer`)
 - `python3-tk` (tkinter)

- Zależności Pythona (patrz `requirements.txt`):

- `python-can`
- `numpy`, `pandas`, `scipy`
- `matplotlib`
- `scikit-learn`
- `torch` (dla LSTM i autoenkodera)
- `canmatrix` (obsługa DBC)
- `pyarrow` (eksport Parquet)
- `asammdf` (eksport MDF4)

4 Instalacja

4.1 Sklonowanie repozytorium

```
git clone https://github.com/twoj_login/can_simulator.  
→ git  
cd can_simulator
```

4.2 Utworzenie środowiska wirtualnego i instalacja zależności

```
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

Jeśli brakuje pliku `requirements.txt`, zainstaluj ręcznie:

```
pip install python-can numpy pandas matplotlib  
→ scikit-learn torch canmatrix pyarrow asammdf
```

4.3 Przygotowanie interfejsu CAN (opcjonalnie wirtualny)

```
sudo modprobe vcan  
sudo ip link add dev vcan0 type vcan  
sudo ip link set up vcan0
```

5 Uruchomienie

```
# Aktywuj środowisko
source .venv/bin/activate

# Uruchom aplikację (wymaga uprawnień do socketów CAN -
  ↳ sudo)
sudo ./run.sh
```

lub bezpośrednio:

```
sudo .venv/bin/python main.py
```

Aplikacja otworzy okno GUI z logiem i panelem bocznym z kategoriami.

6 Struktura projektu

```
can_simulator/
|-- main.py                # Punkt wejścia -
  ↳ logowanie, tworzenie GUI
|-- run.sh                # Skrypt uruchomieniowy (
  ↳ aktywuje venv)
|-- can_interface.py      # Klasa CanInterface - obs
  ↳ ługa SocketCAN
|-- parsers.py            # Parsowanie candump i log
  ↳ ów
|-- session_manager.py    # Zapis/odczyt sesji (JSON
  ↳ )
|-- project_manager.py    # Eksport/import projektu
  ↳ (.csp)
|-- server_mode.py        # Tryb serwera TCP
|-- macro_recorder.py     # Nagrywanie i odtwarzanie
  ↳ makr
|-- controllers/          # Kontrolery operacji CAN
|   |-- base_controller.py
|   |-- can_controller.py
|   |-- replay_controller.py
|   |-- missing_controller.py
|   |-- error_controller.py
|   |-- step_controller.py
|   |-- manual_controller.py
|   |-- binary_controller.py
|   |-- wizard_controller.py
```

```

| | -- sniffer/                                # Kontroler Sniffera (rdze
| |   ↪ ĩ, eksport, DBC)
| |   '-- ecu_emulator.py                      # Emulator ECU (OBD-II)
|-- gui/                                       # Interfejs użytkownika
| | -- app.py                                 # Główna klasa
| |   ↪ CanSimulatorApp
| |   '-- utils.py                            # Funkcje pomocnicze
| |   '-- widgets/                           # Niestandardowe widgety (
| |     ↪ VirtualTreeview)
| |   '-- tabs/                               # Zakładki GUI
| |     |-- replay_tab.py
| |     |-- missing_tab.py
| |     |-- error_tab.py
| |     |-- step_tab.py
| |     |-- manual_tab.py
| |     |-- binary_tab.py
| |     |-- wizard_tab.py
| |     |-- sniffer_tab.py
| |     |-- chart_tab.py
| |     |-- ml_analysis_tab.py
| |     |-- pattern_tab.py
| |     |-- advanced_ml_tab.py
| |     |-- server_tab.py
| |     |-- generator_tab.py
| |     |-- macro_tab.py
| |     |-- console_tab.py                    # Konsola Python
| |     |-- ecu_tab.py                        # Zakładka emulatora ECU
| |     |-- anomaly_tab.py
| |     |-- remote_monitor_tab.py
| |     |-- dbc_manager_tab.py
| |     |-- bridge_tab.py
| |     '-- recording_tab.py
|-- plugins/                                 # System pluginów
| | -- __init__.py
| |   '-- example_plugin.py
|-- tests/                                  # Testy jednostkowe i
| |   ↪ integracyjne
| |   |-- test_parsers.py
| |   |-- test_binary_search.py
| |   |-- test_cyclic_detector.py
| |   |-- test_integration.py
| |   '-- test_wizard.py
'-- ml/                                     # Maszynowe uczenie

```

```
|-- feature_extractor.py
|-- model.py
|-- sequential_model.py
|-- session_classifier.py
|-- pattern_detector.py
'-- advanced_models.py
```

7 Etapy rozwoju

Projekt rozwijał się w etapach:

- **Etap A** – fundamenty: testy jednostkowe, CI (GitHub Actions).
- **Etap B** – wydajność i UX: wirtualny Treeview, dokowanie, motywy.
- **Etap C** – integracja z ekosystemem: eksport Parquet/MDF4, edytor DBC.
- **Etap D** – zaawansowane ML i analiza danych: LSTM, korelacja, eksplorator.
- **Etap E** – funkcje niszowe i społeczność: konsola Python, pluginy, emulator ECU, zdalny eval.

Dodatkowe plany na przyszłość: rozszerzenie testów, strumieniowanie dużych logów, integracja z Vector CANalyzer (.cfg).

8 Testy i CI

Testy jednostkowe znajdują się w katalogu `tests/`. Aby je uruchomić:

```
python3 -m pytest tests/
```

Konfiguracja CI znajduje się w `.github/workflows/test.yml`.

9 Licencja

Projekt udostępniony na licencji MIT – zobacz plik `LICENSE` po więcej informacji.